

Project Development

1. Performance Testing (Theory)

Performance testing is utilized to evaluate how efficiently the system operates under a variety of different conditions. This evaluation helps measure the system speed, responsiveness, and overall stability when tasked with handling large volumes of external data.

In this phase, performance testing is carried out to check the exact efficiency of importing data via Import Sets and the execution speed of Transform Maps. It strictly validates that Dot Walking establishes relationship mapping correctly and without processing delay.

The primary objectives of our performance testing are:

- To analyze and confirm baseline system response times.
- To measure total data processing speeds during heavy loads.
- To ensure continuous system stability when handling large datasets.
- To actively identify bottlenecks in the automated data import process.

Completing this performance check ensures that the system works smoothly even when handling bulk organizational data.

2. Sample Coding (Transform)

To move data correctly from staging files to target files, background scripts handle the lookups automatically. Below is the active example of the JavaScript Transform Map script developed for this project:

JavaScript

```
(function transformRow(source, target, map, log, isUpdate) { //  
    // Copy the target name field //  
    target.name = source.u_name; //
```

```
// Execute Dot Walking (Reference field mapping) //
if (source.u_department) { //
    var dept = new GlideRecord('cmn_department'); //
    dept.addQuery('name', source.u_department); //
    dept.query(); //

    if (dept.next()) { //
        target.department = dept.sys_id; //
    } //
} //
})(source, target, map, log, action === "update"); //
```

Script Breakdown:

- **Source:** Grabs the physical raw data pulled directly from the temporary Import Set.
- **Target:** Points to the pristine data going directly into the final target table.
- **Logic:** The execution script handles mapping your fields automatically.
- **Relational Mapping:** It utilizes a background reference lookup (modeling the Dot Walking concept) to link your records securely.

3. Output & Performance Testing Results

After executing the custom background transform script, the logged outputs yielded the following successful behaviors:

- Data loaded from your external CSV and Excel sheets successfully completed the import sequence.
- Fields successfully mapped directly into the target database tables without failures.
- Department pointers and extended table relationships auto-generated without human intervention.
- No manual corrections or human interactions were required post-execution.
- System execution logs tracked entirely successful operations.

To complete full User Acceptance testing, the system loaded a heavy mock file containing \$1,000\$ dummy records to actively measure pure execution times and log performance behavior under continuous stress.

Visualizing the Development Flowchart (Theory+Structure)

- **Top Box (Input):** Raw Data Source is uploaded.
- **Step 2 (Arrow Down to Process):** Import Set receives and buffers the raw files.
- **Step 3 (Arrow Down to Decision Diamond):** Does standard auto-mapping handle table relations?
- **Step 4 (Arrow Left from "No"):** Trigger GlideRecord Custom Transform Script.
- **Step 5 (Arrow Right from "Yes"):** Direct Table Field Mapping.

- **Step 6 (Both arrows meet at Bottom Target):** Clean Target Table commits finalized records.
- **Step 7 (Final Output Block):** System generates execution logs and testing results.

